Notes:

- HW6 due today (All github classroom)
- HW7 due Dec 4 (All gradescope)
- Final project reports due Dec 4
- Final project schedule posted. Email me slides in **pdf format only** by 9am on the day of your presentation

Last class:

- more on `scipy.optimize`
- looking at timing and profilers: `datetime` library, `cProfile`, `pyinstrument`

In [1]:
```
import estpi
from pyinstrument import Profiler
with Profiler() as profiler:
    estpi.estimate_pi(n=1000000)
profiler.print()
```

```
  _     ._   __/__   _ _  _  _ _/_   Recorded: 10:05:09  Samples:  1252
 /_//_/// /_\ / //_// / //_'/ //     Duration: 1.277     CPU time: 1.243
/   _/                      v5.1.1

Profile at /var/folders/ym/d_bk4s6563gc8fw10symyfg00000gn/T/ipykernel_7662/3544690058.py:3

1.277 <module>  /var/folders/ym/d_bk4s6563gc8fw10symyfg00000gn/T/ipykernel_7662/3544690058.py:1
└─ 1.277 estimate_pi  estpi.py:5
   ├─ 0.860 random_complex_point  estpi.py:24
   │  ├─ 0.481 Random.uniform  random.py:498
   │  │  ├─ 0.335 [self]  random.py
   │  │  └─ 0.146 Random.random  <built-in>
   │  └─ 0.379 [self]  estpi.py
   ├─ 0.239 [self]  estpi.py
   └─ 0.178 inside  estpi.py:17
      ├─ 0.111 [self]  estpi.py
      └─ 0.067 abs  <built-in>
```

How do we make it faster?

Look for what's taking the most time, what's scaling sub-optimally. Here, over 50% of time is in `random_complex_point`, so start there.

Created a new copy of `estpi.py`, called `newestpi.py`

We know that numpy is supposed to provide good tools for working with large arrays, so let's replace `random_complex_point` with a numpy array and native operations on it

In [2]:
```
import newestpi
from pyinstrument import Profiler
with Profiler() as profiler:
    newestpi.estimate_pi(n=1000000)
profiler.print()
```

```
  _     ._   __/__   _ _  _  _ _/_   Recorded: 10:36:53  Samples:  336
 /_//_/// /_\ / //_// / //_'/ //     Duration: 0.362     CPU time: 0.353
/   _/                      v5.1.1

Profile at /var/folders/ym/d_bk4s6563gc8fw10symyfg00000gn/T/ipykernel_7662/2454629531.py:3

0.362 <module>  /var/folders/ym/d_bk4s6563gc8fw10symyfg00000gn/T/ipykernel_7662/2454629531.py:1
└─ 0.362 estimate_pi  newestpi.py:9
   ├─ 0.193 inside  newestpi.py:21
   │  ├─ 0.129 [self]  newestpi.py
   │  └─ 0.064 abs  <built-in>
   ├─ 0.127 [self]  newestpi.py
   └─ 0.042 np_points  newestpi.py:5
      ├─ 0.024 __getattr__  numpy/__init__.py:707
      │     [3 frames hidden]  numpy
      └─ 0.018 [self]  newestpi.py
```

Notes:

- overall time went down, from about 1.2 seconds to 0.36 seconds.
- `np_points` is now least expensive of my routines, taking 0.04 seconds, down from 0.86 seconds
- Now spending about 0.2 seconds in `inside` routine, and 0.1 seconds in self time

How do we reduce these times?

Loop is probably the actual expensive part. How can we reduce that?

Use numpy, and try to rewrite using numpy operations

Work on `inside` first, and use `np.abs` to work on entire array

Simple test:

In [3]:
```
import numpy as np
points = np.random.uniform(size=5) + np.random.uniform(size=5)*1j
```

In [4]:
```
print(points)
```

```
[0.59781586+0.94452729j 0.60520255+0.48107882j 0.96126403+0.98476496j
 0.00097093+0.0619976j  0.07598294+0.41695138j]
```

In [6]:
```
print(np.abs(points))
```

```
[1.11781734 0.7731151  1.37615063 0.0620052  0.42381819]
```

In [7]:
```
print(np.abs(points) < 1)
```

```
[False  True False  True  True]
```

Just use `np.abs(points) < 1` to get true/false values for if a point is inside the unit circle or not

Replace `inside` routine with a numpy-based function

We've changed the module, so we need to reload it in order to profile the new code:

In [8]:
```
import importlib
importlib.reload(newestpi)
with Profiler() as profiler:
    newestpi.estimate_pi(n=1000000)
profiler.print()
```

```
  _     ._   __/__   _ _  _  _ _/_   Recorded: 10:47:32  Samples:  3
 /_//_/// /_\ / //_// / //_'/ //     Duration: 0.060     CPU time: 0.060
/   _/                      v5.1.1

Profile at /var/folders/ym/d_bk4s6563gc8fw10symyfg00000gn/T/ipykernel_7662/2946985028.py:3

0.059 <module>  /var/folders/ym/d_bk4s6563gc8fw10symyfg00000gn/T/ipykernel_7662/2946985028.py:1
└─ 0.059 estimate_pi  newestpi.py:13
   ├─ 0.037 [self]  newestpi.py
   ├─ 0.018 np_points  newestpi.py:5
   └─ 0.005 np_inside  newestpi.py:9
```

Now another 6 times faster. Down to 0.06 seconds. Should not trust timings that are well below 1 second. Could use a smaller interval in `pyinstrument`, or just use a bigger problem

In [9]:
```
import importlib
importlib.reload(newestpi)
with Profiler() as profiler:
    newestpi.estimate_pi(n=10000000)
profiler.print()
```

```
  _     ._   __/__   _ _  _  _ _/_   Recorded: 10:49:22  Samples:  4
 /_//_/// /_\ / //_// / //_'/ //     Duration: 0.551     CPU time: 0.539
/   _/                      v5.1.1

Profile at /var/folders/ym/d_bk4s6563gc8fw10symyfg00000gn/T/ipykernel_7662/472027005.py:3

0.551 <module>  /var/folders/ym/d_bk4s6563gc8fw10symyfg00000gn/T/ipykernel_7662/472027005.py:1
└─ 0.548 estimate_pi  newestpi.py:13
   ├─ 0.350 [self]  newestpi.py
   ├─ 0.164 np_points  newestpi.py:5
   └─ 0.033 np_inside  newestpi.py:9
```

With 10 times the points, taking 0.55 seconds, about 0.35 of which are in `estimate_pi` itself.

Can we eliminate that loop?

In [10]:
```
print(points)
```

```
[0.59781586+0.94452729j 0.60520255+0.48107882j 0.96126403+0.98476496j
 0.00097093+0.0619976j  0.07598294+0.41695138j]
```

In [11]:
```
print(np.abs(points) <= 1)
```

```
[False  True False  True  True]
```

In [12]:
```
print(np.sum(np.abs(points) <= 1))
```

```
3
```

`np.sum` on a True/False array produces the number of True values in the array. This lets us rewrite `estimate_pi` without an explicit loop

In [13]:
```
import importlib
importlib.reload(newestpi)
with Profiler() as profiler:
    newestpi.estimate_pi(n=10000000)
profiler.print()
```

```
  _     ._   __/__   _ _  _  _ _/_   Recorded: 10:53:07  Samples:  4
 /_//_/// /_\ / //_// / //_'/ //     Duration: 0.228     CPU time: 0.202
/   _/                      v5.1.1

Profile at /var/folders/ym/d_bk4s6563gc8fw10symyfg00000gn/T/ipykernel_7662/472027005.py:3

0.227 <module>  /var/folders/ym/d_bk4s6563gc8fw10symyfg00000gn/T/ipykernel_7662/472027005.py:1
├─ 0.224 estimate_pi  newestpi.py:13
│  ├─ 0.187 np_points  newestpi.py:5
│  ├─ 0.035 np_inside  newestpi.py:9
│  └─ 0.002 sum  numpy/_core/fromnumeric.py:2338
│        [2 frames hidden]  numpy, <built-in>
└─ 0.003 [self]  /var/folders/ym/d_bk4s6563gc8fw10symyfg00000gn/T/ipykernel_7662/472027005.py
```

Another almost factor 2 speedup, down to about 0.2 seconds for 10 million points, almost all of which is inside a numpy routine, generating sequence of points.

In [14]:
```
from datetime import datetime
for exponent in range(1,9):
    n = 10**exponent
    start = datetime.now()
    estimates = [newestpi.estimate_pi(n) for _ in range(5)]
    end = datetime.now()
    print(f"{n =:<10,} {estimates} in {(end-start).total_seconds()/5} seconds average time")
```

```
n =10         [np.float64(2.8), np.float64(2.4), np.float64(2.8), np.float64(3.2), np.float64(3.6)] in 0.00011199999999999998 seconds average time
n =100        [np.float64(3.28), np.float64(3.04), np.float64(3.08), np.float64(3.32), np.float64(3.0)] in 3.34e-05 seconds average time
n =1,000      [np.float64(3.116), np.float64(3.144), np.float64(3.096), np.float64(3.128), np.float64(3.14)] in 0.0003038 seconds average time
n =10,000     [np.float64(3.152), np.float64(3.1444), np.float64(3.17), np.float64(3.154), np.float64(3.1732)] in 0.0006196 seconds average time
n =100,000    [np.float64(3.13828), np.float64(3.14088), np.float64(3.14264), np.float64(3.14148), np.float64(3.1386)] in 0.0028358000000000003 seconds average time
n =1,000,000  [np.float64(3.141836), np.float64(3.141852), np.float64(3.13872), np.float64(3.139988), np.float64(3.142104)] in 0.0178358 seconds average time
n =10,000,000 [np.float64(3.1412104), np.float64(3.1407584), np.float64(3.1405924), np.float64(3.1416032), np.float64(3.1418328)] in 0.1585478 seconds average time
n =100,000,000 [np.float64(3.1414922), np.float64(3.14169792), np.float64(3.14156276), np.float64(3.14137428), np.float64(3.14144988)] in 1.9233756 seconds average time
```

Now feasible to run experiments up to n = 100 million, and even bigger if memory allows.

This is a great speedup - about 2 times faster than original code for a problem 10 times bigger.

One more note: in a Jupyter notebook, have another nice timing routine, `%timeit command` that runs a single command in a loop enough times to generate reliable timings

In [15]:
```
n = 10000000
%timeit newestpi.estimate_pi(n)
```

```
153 ms ± 5.19 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

In [16]:
```
n = 10000
%timeit newestpi.estimate_pi(n)
```

```
248 µs ± 6.93 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
```

Can we make it faster still?

Not(*) with more work in Python

Python is what we call an interpreted language. We write source code that is translated into an intermediate representation (called "bytecode") and that code is then executed step-by-step by calls to libraries.

This contrasts to languages like C/C++ or Haskell, where source code is compiled into machine code, and that machine code is executed independently from its compilation

Because compiled languages get compiled directly into machine code, they tend to provide the fastest performance. We can use performant libraries, like Numpy and Scipy, to get best possible performance out of Python codes. But, eventually, we hit a performance barrier that we won't be able to pass without incorporating some compiled code

How do we do this?

3 options:

1. Cython is python compiler that translates python code to C code that can be compiled and called from Python. It allows the use of C/C++ operations within the Python code that is to be compiled, allowing use of C/C++ datatypes and external C/C++ libraries

2. pybind11 provides a tool for creating Python interfaces to C/C++ codes. User writes and compiles C/C++ code for their operations, and uses pybind11 to create "Python bindings" that let us call compiled C/C++ code from Python, with Python datatypes as inputs.

3. `numba`, in contrast, is a python module that provides decorators to allow "Just-in-Time" (JIT) compilation of a subset of Python code, particularly focused on Numpy and other numerical operations

Many scientific computing pacakages make use of Cython or pybind11, particularly to interface with existing C/C++ codes.

For example, PETSc uses Cython to provide python bindings of much of its functionality, which is written in C. If you have a library that provides the tools that you need, this is clearly a good solution.

For many tasks, however, re-implementing existing approaches in C is not an ideal solution - it can be time-consuming, and may not be worth the effort if it is not for critical functionality

This is where numba can be an effective tool. It requires no rewriting, but can still provide effective speedup.

How do we use it?

Start with a simple task: Sum the numbers from 0 to $10^7 - 1$

We know how to do this in 2 ways:

In [17]:
```
sum(range(10**7))
```

Out[17]:
```
49999995000000
```

In [18]:
```
np.sum(np.arange(10**7))
```

Out[18]:
```
np.int64(49999995000000)
```

How efficient are these?

In [19]:
```
%timeit sum(range(10**7))
```

```
120 ms ± 1.16 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

In [20]:
```
%timeit np.sum(np.arange(10**7))
```

```
8.87 ms ± 85.7 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

Using numpt is, as expected, over 10x faster. To use numba for this, we write a regular Python function:

In [21]:
```
def sum_range(n):
    result = 0
    for i in range(n):
        result = result + i
    return result
```

In [22]:
```
sum_range(10**7)
```

Out[22]:
```
49999995000000
```

This gives correct answer. Time it without numba compilation:

In [23]:
```
%timeit sum_range(10**7)
```

```
303 ms ± 8.54 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

Not horrible, but about 2x slower than our slower way of doing this with `sum(range(10**7))`

Now add numba:

In [25]:
```
import numba
@numba.jit(nopython=True)
def sum_range_numba(n):
    result = 0
    for i in range(n):
        result = result + i
    return result
```

In [26]:
```
sum_range_numba(10**7)
```

Out[26]:
```
49999995000000
```

Gets the right answer still, but took a bit of time. That time was not spent running the code, but instead spent compiling the code

In [27]:
```
sum_range_numba(10**7)
```

Out[27]:
```
49999995000000
```

Gets the right answer, and much faster the second time. Now time it:

In [28]:
```
%timeit sum_range_numba(10**7)
```

```
83 ns ± 0.0501 ns per loop (mean ± std. dev. of 7 runs, 10,000,000 loops each)
```

Now takes 80 nanoseconds, a massive speedup over our uncompiled version, and even over numpy

Note: We decorate our functions using `@numpy.jit()`, which stands for just-in-time. We use kwarg `nopython=True` to tell numba to use a compilation that produces faster code (but sometimes fails). An alias for this is `@numba.njit()`, which is equivalent to `@numba.jit(nopython=True)`

Notice: `sum_range_numba` uses loops! We've seen that Python code with loops leads to slow code. This is generally true for interpreted code, but not true for compiled code.

Let's try a comparison:

In [29]:
```
@numba.njit()
def comp_one_np(x):
    return np.cos(x)**2 + np.sin(x)**2

@numba.njit()
def comp_one_loop(x):
    r = np.empty_like(x)
    n = len(x)
    for i in range(n):
        r[i] = np.cos(x[i])**2 + np.sin(x[i])**2
    return r
```

Same calculations, take an array `x`, compute $1 = \sin(x)^2 + \cos(x)^2$ for all entries in the array, either using numpy's native operations or loops.

Use `function.py_func()` to run the original python functions intead of numba-compiled versions

In [30]:
```
x = np.random.uniform(size=10**6)

%timeit comp_one_loop.py_func(x)
```

```
1.08 s ± 1.37 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

In [32]:
```
%timeit comp_one_np.py_func(x)
```

```
10.1 ms ± 402 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

Without compilation, Numpy version is about 100x faster. Try with compiled versions:

In [33]:
```
%timeit comp_one_loop(x)
```

```
6.16 ms ± 39.6 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

In [34]:
```
%timeit comp_one_np(x)
```

```
6.16 ms ± 13.2 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

Performance of both codes is essentially identical. Slightly faster (about 2/3 of the time) as non-compiled Numpy version, but no difference between compiled loop and compile numpy code

Where does Numba work well and work poorly?

- Numba likes code with loops and Numpy functions acting on Numpy arrays
- Numba doesn't know about your own class, or other common modules like Pandas. Can't effectively compile many operations that aren't intensive in numerical computations.

Note: Numba doesn't compile your code when you write the function but, rather, when you first call it. This is what "just-in-time" means. So code will run slowly the first time you call a numba-compiled function, because it is doing the compiling. `%timeit` is usually smart enough to recognize this and exclude a first run from its timings. Sometimes generates a warning

Back to computing π, rewrite original loop from `estimate_pi` to be one self-contained function that we can compile with numba:

In [35]:
```
from random import uniform

@numba.njit()
def estimate_pi_numba(n):
    count_inside = 0
    for i in range(n):
        point = complex(uniform(0,1),uniform(0,1))
        if abs(point) <= 1:
            count_inside += 1
    return 4*count_inside/n
```

First compare without compilation:

In [36]:
```
%timeit estpi.estimate_pi(10**6)
```

```
351 ms ± 723 µs per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

In [37]:
```
%timeit estimate_pi_numba.py_func(10**6)
```

```
319 ms ± 393 µs per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

Roughly same computational time. Now try the compiled version:

In [38]:
```
%timeit newestpi.estimate_pi(10**7)
```

```
151 ms ± 1.21 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

In [39]:
```
%timeit estimate_pi_numba(10**7)
```

```
128 ms ± 170 µs per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

Get small improvement on numpy implementation, consistent with our experiment above to time computing $\sin^2(x) + \cos^2(x)$

Has another advantage: out numpy implementation requires us to generate and store one big array of all of the complex numbers we want to use in our computation. Eventually that requires too much memory. Loop version generates points one at a time, so has a much lower memory cost.

In [40]:
```
from datetime import datetime
for exponent in range(1,10):
    n = 10**exponent
    start = datetime.now()
    estimates = [estimate_pi_numba(n) for _ in range(5)]
    end = datetime.now()
    print(f"{n =:<10,} {estimates} in {(end-start).total_seconds()/5} seconds average time")
```

```
n =10         [2.8, 3.6, 2.4, 3.2, 2.8] in 1.4200000000000001e-05 seconds average time
n =100        [3.16, 3.2, 3.12, 3.2, 3.08] in 3.2e-06 seconds average time
n =1,000      [3.172, 3.14, 3.212, 3.124, 3.104] in 1.56e-05 seconds average time
n =10,000     [3.1472, 3.1488, 3.126, 3.1236, 3.1476] in 0.0001618 seconds average time
n =100,000    [3.14588, 3.14016, 3.14488, 3.13648, 3.13492] in 0.001626 seconds average time
n =1,000,000  [3.142516, 3.140684, 3.14184, 3.141988, 3.14108] in 0.0138432 seconds average time
n =10,000,000 [3.1419912, 3.1416888, 3.1407004, 3.1414912, 3.1423944] in 0.1349878 seconds average time
n =100,000,000 [3.14173308, 3.141369, 3.141878, 3.14128724, 3.14134812] in 1.2798756 seconds average time
n =1,000,000,000 [3.141664616, 3.141521496, 3.14164516, 3.141570056, 3.141610744] in 12.7793448 seconds average time
n =1,000,000,000 [3.141664616, 3.141521496, 3.14164516, 3.141570056, 3.141610744] in 12.7793448 seconds average time
```

Now able to compute up to n = 1 billion reliably, in ~13 seconds average time

Numba also has some ability to parallelize code as it compiles, although doesn't do much for this code (at least on my laptop, as it's written)

Other options to parallelize Python include the `multiprocessing` module from the Python standard library, or the `mpi4py` package that exposes a standard MPI communicator and the gather/scatter operations to us from within Python

Another option is the Dask library that gives parallel extensions of Numpy and Pandas datatypes, as well as tools for dynamic task scheduling on parallel machines